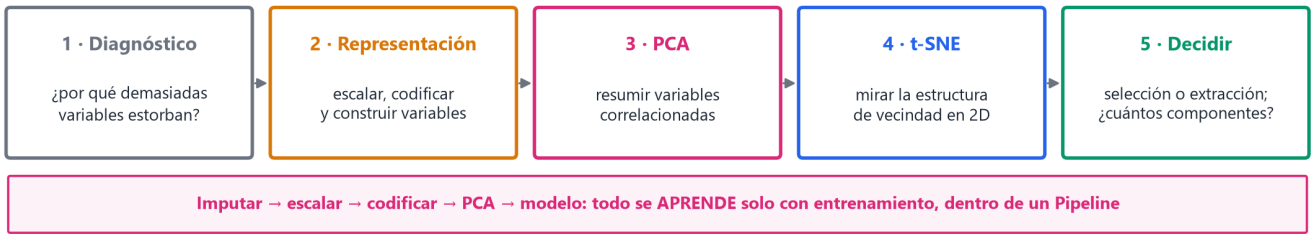
Clase 32 · Reducción de dimensionalidad y representación de variables

Profesora Laura Sánchez · bloque de Machine Learning · sigue a Preparación para ML (C30) e Introducción al ML (C31)

La gran idea de la clase

Un modelo no recibe «datos»: recibe una **representación**. Antes de modelar hay que **escalar**, **codificar** y **construir** variables, y cuando hay demasiadas columnas (redundantes o ruidosas) conviene **resumirlas**. **PCA** las resume en pocas combinaciones lineales que conservan la mayor varianza; **t-SNE** dibuja en 2D quién es vecino de quién para **explorar**, no para producir variables. «Las matemáticas buscan el camino más eficiente: reducir dimensiones es eficiente, pero no es reducir por reducir.»

La clase en una imagen



Lo que necesitas ANTES

- Correlación, varianza y desviación estándar (Etapa 0); regresión múltiple y multicolinealidad (Clases 17–19)
- Train / validación / prueba, fuga de datos y Pipeline (Clase 30)
- Supervisado vs no supervisado: PCA y t-SNE no usan la etiqueta (Clase 31)
- Vectores y matrices: basta la idea de «dirección» y «proporcional» (la profesora evita las fórmulas)

Ruta de la clase y datos

- **Diagnóstico** → **representación** (escalar, codificar, construir) → **PCA** (varianza explicada, cargas) → **t-SNE** (perplexity) → decidir
- Datos: **1.500 clientes de telco simulados** (el notebook de la profesora los genera con `default_rng(32)`, 15 % de fuga) y la **base real de 3.150 clientes** de la actividad de repaso (15.7 % de fuga)
- Pregunta guía: ¿cuántas dimensiones necesito de verdad y cómo las entrego al modelo sin hacer trampa?

Lo mínimo de scikit-learn para hoy

Los tres verbos de la Clase 30 (`fit`, `transform`, `predict`) valen igual para `StandardScaler` y `PCA`: ambos **aprenden** con `fit(X_train)`. PCA tiene además `explained_variance_ratio_` (varianza explicada) y `components_` (las cargas). Todo va dentro de un `Pipeline` (página 7). Sobre la API: Clase 30, página 7.

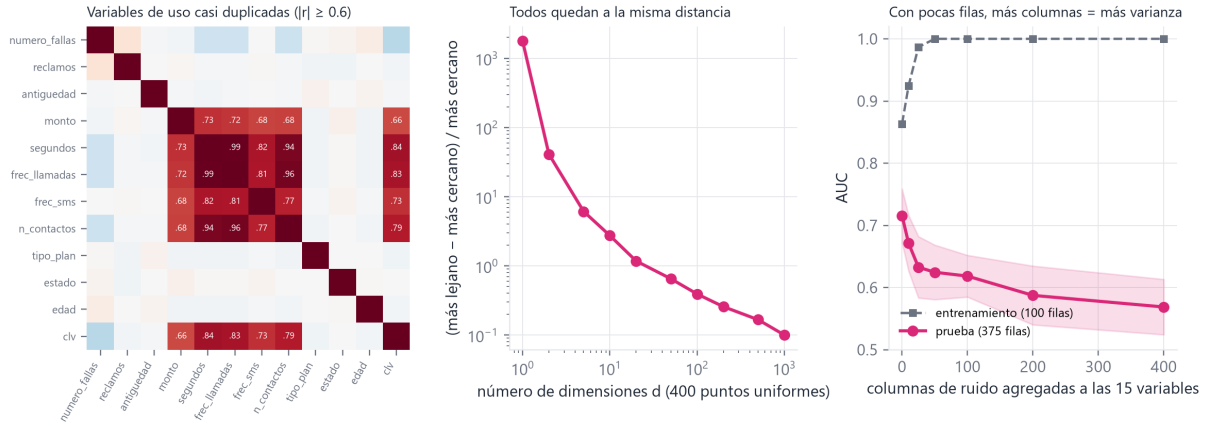
Clase 32 · El problema de la dimensionalidad

Fuente: transcripción de la clase + diapositivas + apunte + notebook y caso de la profesora Laura Sánchez

Más columnas no son más información. Las variables sobran por cuatro razones: costo, multicolinealidad, ruido y sesgo vs. varianza. Se reduce por selección (quitar columnas) o por extracción (combinarlas).

Conceptos clave

1. **Costo computacional.** Más columnas, más memoria y más tiempo de entrenamiento; más difícil de poner en producción, depurar y explicar.
2. **Multicolinealidad.** Variables que cuentan la misma historia. En el notebook, `segundos` y `frec_llamadas` tienen r = 0.99; el VIF de ambas supera 40 (regla práctica: > 10 es problema).
3. **Variables ruido.** Columnas sin señal. Con 100 filas, agregar 400 columnas de ruido baja el AUC en prueba de 0.72 a 0.57 mientras en entrenamiento llega a 1.00: sobreajuste.
4. **Sesgo vs. varianza.** Con pocas observaciones, más variables aumentan la varianza de las predicciones sin compensar el sesgo; además, en dimensiones altas las distancias se «aplanan».



Izquierda: correlaciones de las 12 variables originales (se rotulan |r| ≥ 0.6): seis variables de uso casi duplicadas. Centro: con puntos uniformes, la razón (más lejano − más cercano)/más cercano cae de 41 (d = 2) a 0.1 (d = 1.000). Derecha: AUC de una regresión logística con 100 filas al agregar columnas de ruido (25 repeticiones).

Fórmulas y reglas

$\text{VIF}_j = 1 / (1 - R^2_j)$

filas por columna = n / p

selección: quita columnas · extracción: las combina

En Python

```
df.corr() · df.shape
# VIF: 1/(1-R²) con LinearRegression
X = df.drop(columns='fuga')
```

Con los datos de la clase

- 1.500 clientes, 12 variables, fuga 15.07 %; las 6 de uso correlacionan entre 0.66 y 0.99
- VIF: segundos 40.8, frec_llamadas 48.3, n_contactos 12.2
- «No hay un número ideal» de variables (profesora)

Cuidado
Correlacionadas no es lo mismo que ruido: el ruido no aporta nada aunque no se parezca a ninguna.

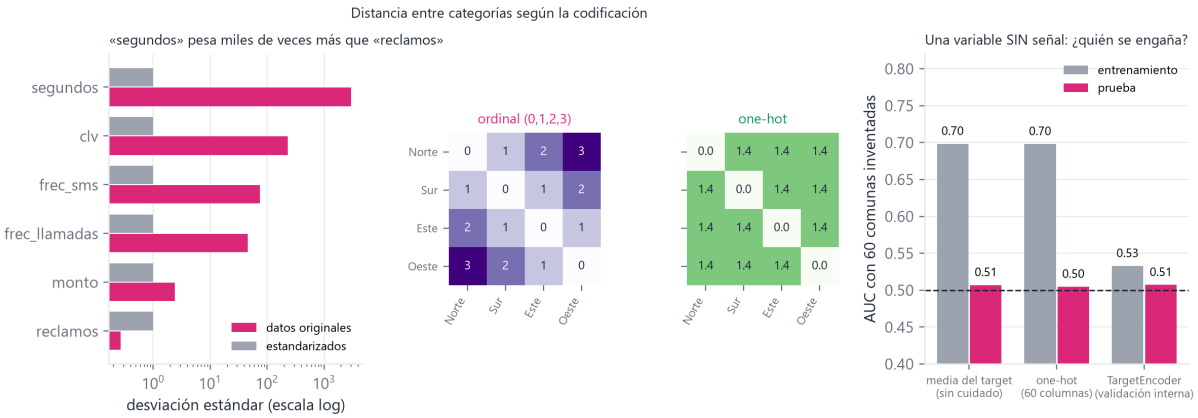
Clase 32 · Representación de variables: escalar, codificar y construir

Fuente: transcripción de la clase + diapositivas + apunte + notebook y caso de la profesora Laura Sánchez

La Clase 30 preguntó «¿son confiables los datos?». Hoy: «¿en qué forma se los entrego al modelo?». Un dato correcto pero mal representado hace que PCA o t-SNE resuman una distorsión.

Conceptos clave

1. **Escalas distintas.** `segundos` llega a miles y `reclamos` es 0 o 1. Sin escalar, la variable de mayor magnitud domina varianzas y distancias.
2. **Estandarizar vs. normalizar. Estandarización** (z-score): cuántas desviaciones se aleja de la media; centro 0 y desviación 1; es lo habitual para PCA. **Min-max**: comprime a [0, 1]. No hace falta que la variable sea normal.
3. **Codificar categóricas. Ordinal**: enteros cuando hay orden real (monto 0 a 9). **One-hot**: una columna 0/1 por categoría, sin imponer orden. **Target / frequency**: la categoría se reemplaza por una tasa o una frecuencia; sirve con muchas categorías.
4. **Construir variables.** La señal puede estar en un cociente: `segundos / frec_llamadas`, `frec_sms / antiguedad`, `n_contactos / frec_llamadas`. Se crean **antes** de reducir.



Izquierda: desviación estándar de seis variables (escala log): 2.969 para segundos y 0.27 para reclamos. Centro: con códigos 0, 1, 2, 3, «Norte» queda a 3 de «Oeste» y a 1 de «Sur» sin motivo; con one-hot todas distan √2 = 1.41. Derecha: 60 comunas inventadas, sin relación con la fuga: la media del target y el one-hot «logran» 0.70 de AUC en entrenamiento y 0.51 en prueba.

Fórmulas y reglas

z = (x − media) / desviación estándar

min-max = (x − mín) / (máx − mín)

one-hot: 1 columna por categoría (drop evita una)

En Python

```
StandardScaler().fit(X_train)
OneHotEncoder(drop='if_binary')
TargetEncoder(cv=5) # tasa por categoría
```

Con los datos de la clase

- Variables construidas: 12 → 15 columnas; one-hot de tipo_plan y estado (drop if_binary): sigue en 15
- Target encoding «a mano», 60 categorías sin señal: AUC 0.70 en entrenamiento, 0.51 en prueba; `TargetEncoder`: 0.53 y 0.51

Cuidado
La diapositiva dice «Ordinal: Nominal → entero»; es para variables **ordinales**. El target encoding con todas las filas es fuga.

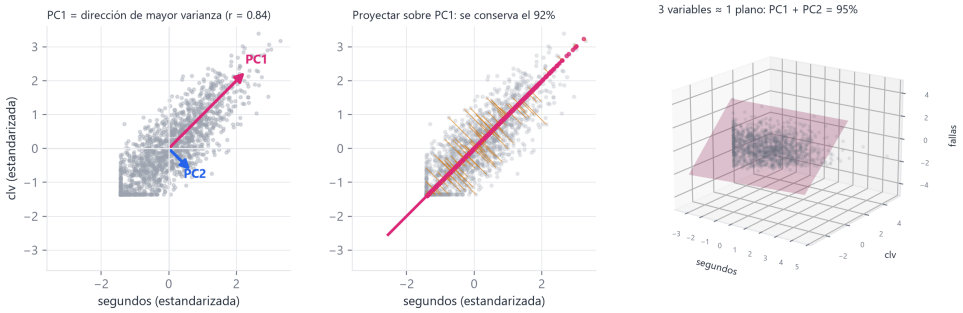
Clase 32 · PCA: la idea, la varianza explicada y cuántos componentes

Fuente: transcripción de la clase + diapositivas + apunte + notebook y caso de la profesora Laura Sánchez

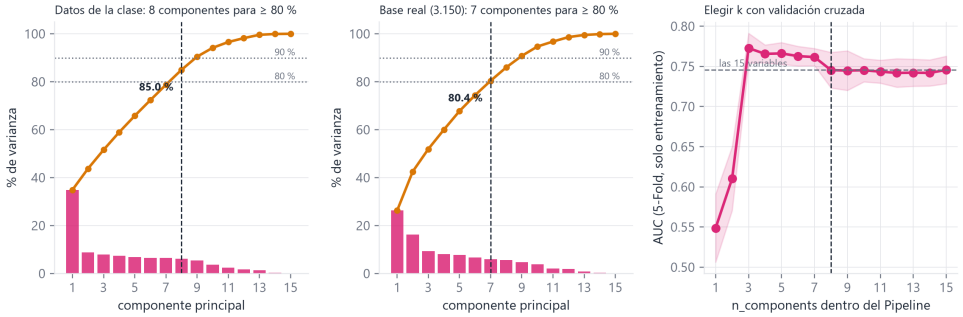
PCA busca la dirección en que los datos varían más (PC1), luego la siguiente dirección perpendicular (PC2), etc. Cada componente es una receta (combinación lineal) y no usa la etiqueta. Se eligen los primeros k que acumulan suficiente varianza.

Conceptos clave

1. **Direcciones ortogonales.** Los componentes son los **vectores propios** de la matriz de covarianza de los datos estandarizados; el **valor propio** (λ) es la varianza que captura cada uno. Se ordenan de mayor a menor.
2. **Varianza explicada.** `explained_variance_ratio_` = λ / Σλ. Con p variables estandarizadas, Σλ = p. El gráfico de barras es el scree plot y la línea, la acumulada.
3. **¿Cuántos componentes?.** No hay un número mágico: se busca el codo, un umbral (80 o 90 %) o, mejor, el desempeño del modelo con validación cruzada dentro de un Pipeline.
4. **No supervisado.** Varianza no es relevancia: un componente chico puede ser el que separa la fuga.



Con `segundos` y `clv` estandarizadas (r = 0.84): PC1 tiene λ = 1.84 y PC2 λ = 0.16. Proyectar sobre PC1 conserva el 91.8 % de la varianza. Derecha: tres variables que casi caben en un plano (PC1 + PC2 = 95 %).



Scree plot de los datos de la clase (8 componentes = 85.0 %), de la base real (7 = 80.4 %) y AUC con validación cruzada de un Pipeline (escalar → PCA → logística) según k.

Fórmulas y reglas

$\Sigma v = \lambda v \cdot \lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_p$

varianza explicada de j = $\lambda_j / \Sigma \lambda$

puntaje = Σ carga$_j \cdot z_j$

Con los datos de la clase

- Con 7 componentes 78.7 %; con 8, 85.0 %; con 9, 90.5 %. Base real: 7 → 80.4 %
- Logística en prueba: 15 variables F1 0.270 / MCC 0.265; PCA 8: 0.267 / 0.252. Base real: 0.574 / 0.546 vs PCA 7: 0.548 / 0.534

Cuidado
AUC en 5-Fold: 3 componentes 0.773 > 15 variables 0.746; F1 0.270 vs 0.267 es ruido (57 fugas en la prueba).

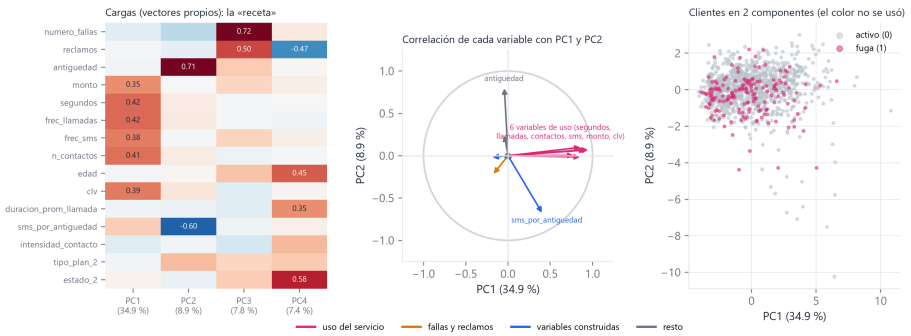
Clase 32 · Leer los componentes: cargas y escala

Fuente: transcripción de la clase + diapositivas + apunte + notebook y caso de la profesora Laura Sánchez

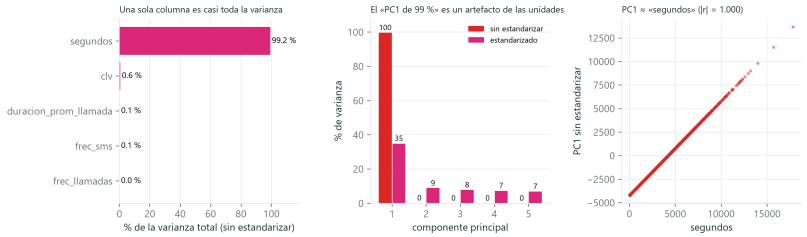
Una carga dice cuánto pesa cada variable en un componente. Se lee con cuidado: el signo de un componente es arbitrario, y sin estandarizar el primer componente solo refleja la unidad de la variable más grande.

Conceptos clave

1. **Cargas.** Cada fila de `components_` tiene norma 1. Las cargas grandes (en valor absoluto) definen de qué trata el componente.
2. **Círculo de correlaciones.** Con datos estandarizados, carga × √λ es la **correlación** entre la variable y el componente. Flechas largas y juntas: variables redundantes.
3. **Interpretar con cautela.** PC1 = «uso del servicio». PC2 es antigüedad y SMS por antigüedad; fallas y reclamos están en PC3. El nombre lo pone quien analiza.
4. **Sin estandarizar.** PCA maximiza varianza: la variable en miles se lleva casi todo; el resultado refleja las unidades.



Cargas de PC1 a PC4, círculo de correlaciones y clientes en PC1–PC2 (el color de la fuga no se usó para construirlos: PC1–PC2 no la separan).



Sin estandarizar: «segundos» concentra el 99.2 % de la varianza, PC1 explica 99.71 % y coincide con «segundos» (|r| ≈ 1.000).

Fórmulas y reglas

correlación(variable, PC_j) = $carga_j \cdot \sqrt{\lambda_j}$

norma de un componente = 1; son □ entre sí

k componentes □ n × k puntajes

Con los datos de la clase

- PC1 (34.9 %): segundos, llamadas, contactos, CLV, SMS (0.38 a 0.42) · PC2 (8.9 %): antiguedad 0.71, sms_por_antiguedad −0.60
- PC3 (7.8 %): numero_fallas 0.72, reclamos 0.50; correlación con la fuga 0.33 (PC1: −0.06)
- Sin estandarizar, el 90 % de la varianza se alcanza con 1 componente

Cuidado
La diapositiva pone «fallas y reclamos» en PC2; en el notebook son PC3. Comprueba con `components_`.

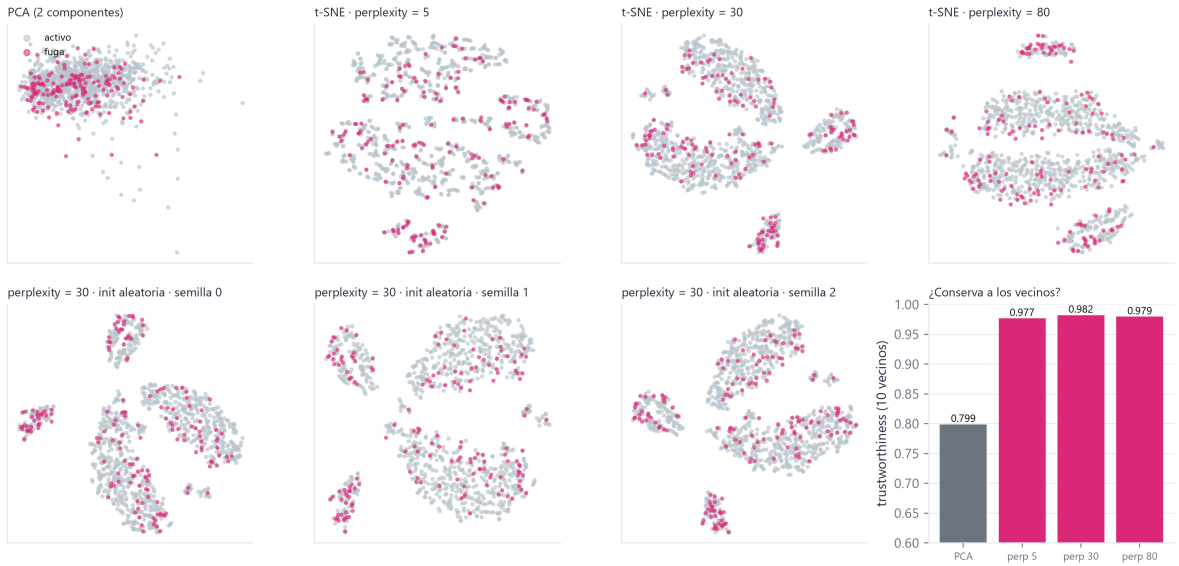
Clase 32 · t-SNE: un mapa no lineal para explorar

Fuente: transcripción de la clase + diapositivas + apunte + notebook y caso de la profesora Laura Sánchez

t-SNE dibuja los clientes en 2D de modo que los vecinos del espacio original queden cerca. Sirve para mirar estructura; no es una transformación para producción ni un algoritmo de clústeres.

Conceptos clave

1. **Qué conserva.** PCA conserva la varianza global (lineal y determinista). t-SNE conserva vecindades locales (no lineal); las distancias entre grupos y el tamaño de las islas no se interpretan.
2. **Perplexity.** Aproximadamente «a cuántos vecinos le presto atención»: baja → grupos chicos y locales; alta → vista más global. Se prueba entre 5 y 50 y debe ser menor que n.
3. **No determinista, sin transform.** Otra semilla, otro dibujo; `TSNE` solo tiene `fit_transform` (no proyecta clientes nuevos) y es costoso. Modelar o producir: PCA. Explorar: t-SNE.



Fila 1: PCA en 2D y t-SNE con perplexity 5, 30 y 80 sobre el entrenamiento (1.125 clientes). Fila 2: tres semillas distintas con perplexity 30 y la «trustworthiness» (qué tanto se conservan los 10 vecinos más cercanos).

Fórmulas y reglas

trustworthiness ∈ [0, 1]: 1 = vecinos conservados

perplexity < n_observaciones

t-SNE no es K-means: no hay centroides

En Python

```
TSNE(perplexity=30, init='pca',
     random_state=32).fit_transform(X)
trustworthiness(X, emb, n_neighbors=10)
```

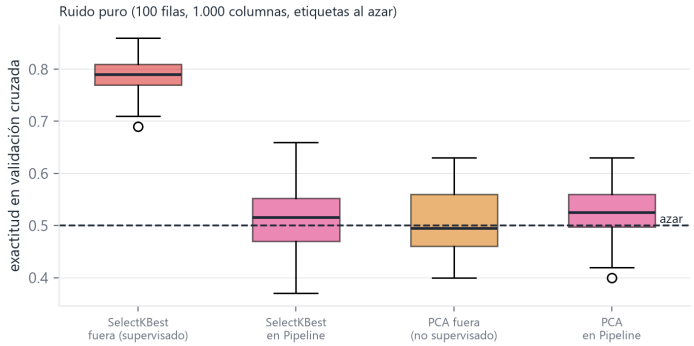
Con los datos de la clase

- Trustworthiness (10 vecinos): PCA 2D 0.799; t-SNE 0.977 (perplexity 5), 0.982 (30), 0.979 (80); 2–4 s por ajuste
- Perplexity 5 parte los grupos en piezas; con otra semilla cambia la posición de las islas
- La fuga no forma un grupo propio: t-SNE no usa la etiqueta

Cuidado
Ver «islas» no prueba que existan clústeres; tamaño y distancia entre islas no se interpretan.

Pipeline, fuga de datos y cómo decidir

Todo lo que aprende de los datos se ajusta solo con entrenamiento



MAL ajustar scaler / imputador / PCA / encoder con todas las filas → **luego dividir o validar**

BIEN dividir primero; Pipeline.fit(X_train, y_train) → **cada paso aprende solo con train (y con cada pliegue)**

PCA no ve «y», por eso su fuga es discreta; la selección y el target encoding sí ven «y».

Datos de puro ruido (100 filas, 1.000 columnas, etiquetas al azar; 40 repeticiones): seleccionar las 10 «mejores» con todas las filas y luego validar da ≈ 0.79 de exactitud; en un Pipeline, ≈ 0.51. PCA ajustado fuera de la validación da ≈ 0.51 (igual que dentro, 0.52): no ve la etiqueta, así que no la filtra, pero igual contamina (la prueba influye en los componentes) y el hábito es peligroso.

Paso	Qué aprende de los datos	Dónde va
Imputador (mediana, moda)	la mediana / la moda	Pipeline, fit con entrenamiento
StandardScaler	media y desviación por columna	Pipeline, fit con entrenamiento
OneHotEncoder / OrdinalEncoder	la lista de categorías	ColumnTransformer dentro del Pipeline
TargetEncoder	la tasa del target por categoría (mira «y»)	Pipeline; usa validación cruzada interna
PCA	los componentes (vectores propios)	Pipeline, después del escalador
t-SNE	no tiene transform	solo para explorar, fuera del modelo

El Pipeline de la clase, honesto

```
make_pipeline(StandardScaler(), PCA(n_components=8), LogisticRegression()).fit(X_train, y_train)
```

Con `GridSearchCV` o `cross_val_score`, el escalador y el PCA se reajustan en cada pliegue: así se elige k sin mirar la prueba.

	Checklist del flujo	Cómo sé que lo hice bien
☐	Construyo variables antes de reducir	los cocientes no se obtienen desde los componentes
☐	Divido antes de escalar	el scaler y el PCA se ajustan solo con entrenamiento
☐	Estandarizo antes de PCA	si no, PC1 es la variable de mayor unidad
☐	Elijo k con criterio y con validación	umbral de varianza, codo y AUC/F1 en CV
☐	Interpreto los componentes con sus cargas	no con el nombre de la diapositiva
☐	Uso t-SNE solo para explorar	no como entrada de un modelo en producción

Alertas del material, repaso y criterio de dominio

Diferencias entre materiales y qué conviene corregir (verificado con código)

Diapositivas, apunte y notebook

- **«7 componentes (≈ 80 %)»** (diapositiva 21) pero la tabla dice 8. Con los datos del notebook: 7 componentes = 78.7 % (no llega a 80), 8 = 85.0 %. En la base real, 7 = 80.4 %.
- **PC2 = fallas y reclamos** (diapositiva 19 y apunte): en el notebook PC2 es antigüedad (0.71) y sms_por_antiguedad (−0.60); fallas (0.72) y reclamos (0.50) forman PC3. En la base real PC2 es SMS, CLV y sms_por_antiguedad.
- **«Dataset real de 1.500 clientes»** (diapositiva 16): los 1.500 son sintéticos; la base real tiene 3.150. **USE_REAL_DATA = True** falla: `RUTA_CHURN` no está definida y se lee con `read_csv(sep=';')` un archivo .xlsx.
- **Ejemplo de valores propios**: usa la matriz no simétrica [[10, −18], [6, −11]] (autovalores 1 y −2, vectores con producto 0.99, no ortogonales). PCA diagonaliza una matriz de **covarianzas, simétrica**: sus vectores son ortogonales (`np.linalg.eigh`).
- **Cuánto cuesta PCA**: F1 0.270 vs 0.267 con 57 fugas en la prueba no es una diferencia; con AUC en 5-Fold, 3 componentes (0.773 ± 0.019) superan a las 15 variables (0.746 ± 0.017) y 8 empatan (0.745). «Se cede algo de desempeño» no es una regla.
- **Ordinal encoding**: la diapositiva dice «Nominal → entero» y en clase «es para las nominales»; es para variables **ordinales**. **Target / frequency encoding** son dos técnicas, y la primera exige validación interna (0.70 → 0.51 en el experimento).
- **Limpieza de la base real**: el código de la clase solo corrige edad y faltantes; quedan negativos imposibles (segundos hasta −14.645, frec_llamadas −176…) y edades 0–14; además monto llega a 10 (7 filas) y la diapositiva dice 0 a 9.

Lo dicho en clase

- «**PCA es un clustering**»: no; PCA es una proyección lineal sin grupos. «**El 85 % de fuga**… con 8 componentes cubrimos a los que se fugan»: el 85 % de varianza y el 85 % de clientes que NO se fugan son cantidades sin relación (la fuga es 15 %).
- **Varianza** como «promedio de distancias a la media»: son distancias al **cuadrado**. Y desbalance de clases no es el «sesgo» del par sesgo-varianza.
- «Con train/test sale la misma descomposición que con tres conjuntos»: no; PCA depende de las filas con que se ajusta.
- **Caso de repaso, pregunta 3.2**: dice que fallas y reclamos apuntan opuestos al uso. En la base real, fallas se correlaciona **positivamente** con segundos (+0.49), llamadas (+0.57) y monto (+0.59; el propio caso lo reporta), carga +0.61 en PC1 vs 0.84 de segundos; solo reclamos es ligeramente negativo (−0.12). La interpretación «más fallas, menos uso» no se sostiene.
- **Hinton & van der Maaten** (2008): el orden de autores es van der Maaten y Hinton.

Transcripción con ruido

Whisper confundió t-SNE (TSN, TSM, TESEC, DCNE), PCA (pcea, pcl, PFA), Lasso (laso, lazo), Ridge (reach), Feature Engineering (Future), K-means (camins), Matthews (Mathieu's), Ñuñoa (yu-nyua) y eigenvalues (agent values). Hubo bucles de repetición (ya omitidos) y el recreo (≈ 1:03:29) no está en el video. Frases dudosas: «la paridad de clientes» (¿fuga?), «para qué voy a hacer todos el astre», «la que anita en la RAE».

Criterio de dominio para pasar a la siguiente clase

- ☐ Calculo a mano la varianza explicada desde los valores propios y elijo k para un umbral
- ☐ Diagonalizo una matriz de covarianza 2×2 y proyecto un punto
- ☐ Leo cargas y escribo qué significa un componente sin inventar
- ☐ Elijo ordinal, one-hot o target encoding según el tipo de variable
- ☐ Escribo un Pipeline con ColumnTransformer, StandardScaler y PCA sin fuga
- ☐ Resuelvo los bloques 1 a 7 del cuaderno con al menos 80 % de aciertos